

Arquitectura de la Aplicación

(A) Escalabilidad Capa Web

La arquitectura diseñada para la capa web en Google Cloud Platform (GCP) está compuesta por una VPC única que alberga tres subredes especializadas, cada una con funciones específicas para garantizar escalabilidad, resiliencia y alto desempeño.

La primera subred (10.109.1.32/27) está dedicada a la instancia worker, que ejecuta microservicios clave para el procesamiento de datos. Estos microservicios se encargan de leer y transformar los documentos cargados por los usuarios, insertar la información en una base de datos vectorial (PostgreSQL) administrada mediante Cloud SQL (PaaS), y comunicarse con el modelo Gemini para el procesamiento avanzado de consultas. Esta instancia actúa como el núcleo de procesamiento de la arquitectura, asegurando que los datos sean correctamente estructurados y almacenados para su posterior recuperación.

En la segunda subred (10.109.1.0/27), se despliega un grupo de autoescalado (instance group) que aloja los microservicios del frontend y backend (web-server). Este grupo gestiona las solicitudes de los usuarios, autenticándolas mediante una base de datos relacional y redirigiéndolas a la instancia worker para su procesamiento. Además, se encarga de almacenar los documentos en Cloud Storage y coordinar el flujo de chats y archivos hacia los workers. Para manejar cargas variables, el grupo emplea una política de autoescalado que inicia con una sola instancia y escala hasta tres cuando el uso de CPU supera el 60%, optimizando así el rendimiento y la disponibilidad del servicio.

La tercera subred (10.0.0.0/26) funciona como capa de proxy y alberga un balanceador de carga, que es el punto de entrada principal para las solicitudes externas. Este componente verifica constantemente el estado de las instancias en el grupo de autoescalado y distribuye el tráfico únicamente hacia aquellas que están activas y saludables, mejorando la confiabilidad del sistema.

La arquitectura se complementa con servicios administrados de GCP que potencian su funcionalidad. El Artifact Registry centraliza las imágenes Docker utilizadas en los despliegues, agilizando la gestión de contenedores. Por otro lado, Gemini proporciona capacidades avanzadas de procesamiento de lenguaje natural (NLP), mientras que Cloud SQL soporta tanto la base de datos relacional para autenticación como la base de datos vectorial para búsquedas semánticas. Finalmente, Cloud Storage ofrece un almacenamiento seguro y escalable para los documentos subidos por los usuarios. La Figura 1 representa la arquitectura descrita.

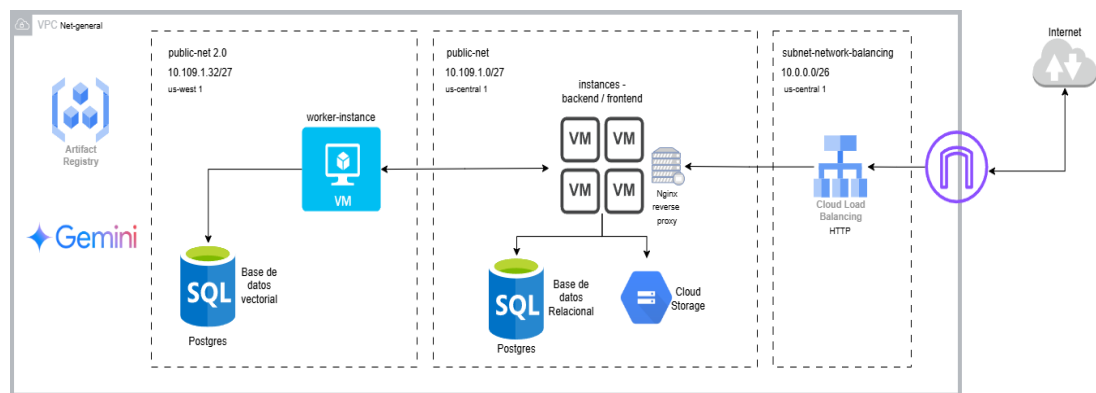


Figura 1. Modelo Despliegue – Escalabilidad capa Web

(B) Escalabilidad en el Backend (workers)

Como parte de la evolución del proyecto y para establecer un punto de comparación en las pruebas de rendimiento, se implementaron cambios significativos sobre la arquitectura inicial de escalabilidad de la capa web. Estas modificaciones buscan mejorar la capacidad de procesamiento y distribuir eficientemente la carga de trabajo.

El primer cambio fundamental fue la migración de la infraestructura de workers a una nueva subred (10.109.2.0/27) en la región us-east5. Esta decisión se tomó debido a limitaciones en el número de instancias permitidas en la subred original (10.109.1.32/27) de la región us-west1. La nueva configuración implementa un grupo de instancias autoescalables (instance group) para los workers, similar al utilizado en el web-server de la arquitectura inicial. Este grupo aplica la misma política de autoescalado, expandiéndose cuando la utilización de CPU supera el 60%, lo que proporciona mayor capacidad operativa y resiliencia durante picos de demanda.

Como segunda mejora, se incorporó el servicio Pub/Sub de GCP, que actúa como sistema de colas y balanceador de carga inteligente entre la capa web (frontend/backend) y la capa de procesamiento (workers). Esta solución ofrece múltiples ventajas:

1. Funciona como buffer para las solicitudes, evitando la sobrecarga de los microservicios.
2. Garantiza la persistencia de las peticiones incluso durante escalamientos o fallos temporales.
3. Distribuye automáticamente la carga entre los workers disponibles
4. Permite comunicación bidireccional eficiente entre componentes

La implementación de Pub/Sub no solo optimiza el flujo de trabajo, sino que también desacopla los componentes del sistema, mejorando la mantenibilidad y permitiendo un escalamiento independiente de cada capa. Esta arquitectura revisada proporciona una base más robusta para manejar cargas variables y mantiene la coherencia con los servicios gestionados de GCP utilizados en el diseño original. La Figura 2 muestra el cambio descrito.

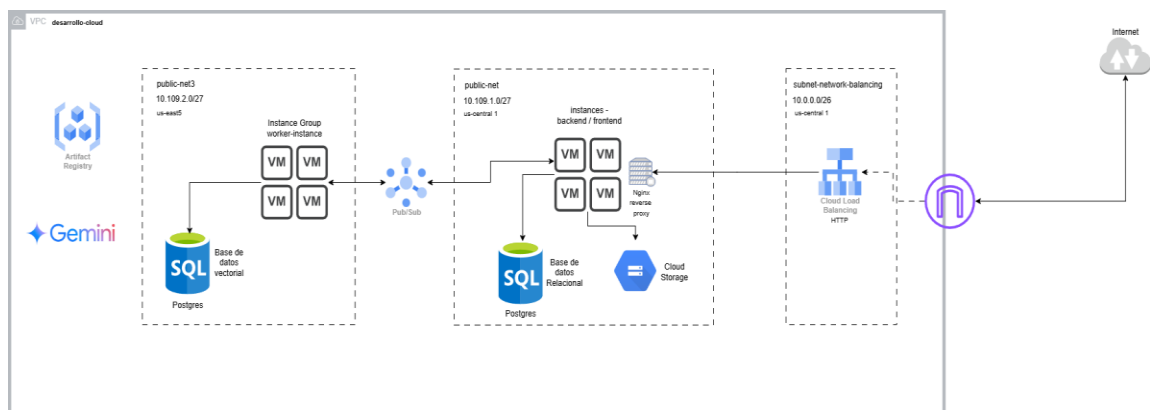


Figura 2. Modelo Despliegue – Escalabilidad capa de proceso (workers)

Conclusiones de pruebas de carga

Durante el proceso de pruebas de carga, se identificó un rendimiento sustancial en los microservicios que conforman la aplicación. En comparación con la entrega anterior, se observó una mejora significativa en los servicios de la capa web-server, especialmente en la Arquitectura A (escalabilidad de la capa web-server). Mientras que en la segunda entrega el sistema alcanzaba su límite con aproximadamente 70 usuarios, en esta arquitectura, se

alcanzó un umbral de 257 usuarios interactuando con la ruta crítica (desde el login hasta la carga del documento) antes de que se presentara degradación. Las pruebas se realizaron con 300 usuarios en 240 segundos, lo que permitió concluir que el balanceo de carga y las políticas de autoescalado fueron clave para evitar un colapso prematuro del sistema. Además, se logró atender entre 3 y 6 clientes simultáneamente en ciertos intervalos de tiempo, demostrando una capacidad de respuesta eficiente.

Por su parte, la Arquitectura B (escalado de la capa de proceso) mostró un rendimiento aún más destacado. Las pruebas se ejecutaron con 600 usuarios en 240 segundos, de los cuales 397 completaron exitosamente la ruta crítica 1, casi el doble que en la Arquitectura A. Además, se observó una atención recurrente de 8 a 12 clientes simultáneos en determinados periodos, lo que confirma una mayor capacidad de procesamiento.

Un hallazgo relevante fue que la capa de proceso (workers) en ambas arquitecturas no experimentó sobrecarga, permitiendo manejar la demanda de clientes que completaron la ruta crítica (257 en la Arquitectura A y 397 en la B). Esto indica que el cuello de botella se encontraba en la capa web-server, ya que los workers nunca alcanzaron su límite de capacidad. Los picos máximos de CPU en esta capa fueron del 38.96% (Arquitectura A) y 197% (Arquitectura B), muy por debajo del máximo teórico del 300%.

Al igual que en la entrega anterior, los servicios administrados (Cloud SQL y Cloud Storage) demostraron alta disponibilidad y adaptabilidad ante el incremento de demanda, sin convertirse en puntos únicos de fallo.

Los resultados evidencian que las nuevas arquitecturas marcaron un hito en las pruebas de carga, logrando una mejora superior al 200% en rendimiento. Este avance se atribuye directamente a las estrategias de escalabilidad implementadas, consolidando un sistema más robusto y eficiente.

Consideraciones adicionales

La implementación del balanceador de carga combinado con Pub/Sub como sistema de colas demostró ser estratégicamente acertada, evidenciándose una mejora sustancial en la capacidad del sistema. Esta arquitectura permitió manejar un mayor volumen de solicitudes concurrentes y soportar un crecimiento significativo de usuarios recurrentes sin comprometer la estabilidad del servicio.

Sin embargo, durante las pruebas de carga identificamos una limitación crítica en el mecanismo de escalamiento actual. Los grupos de instancias presentan una latencia considerable en su respuesta a picos de demanda, debido principalmente al tiempo requerido para el aprovisionamiento y la inicialización de nuevas instancias (incluyendo el arranque de los microservicios). Para mitigar este problema, proponemos dos estrategias complementarias:

1. Optimización del autoescalado tradicional:

- Mantener instancias reservadas desde el inicio (aunque pueda generar cierta subutilización de recursos)
- Ajustar los umbrales de escalado a valores más conservadores (40%-50% de uso de CPU)
- Esto proporciona un margen temporal suficiente para que nuevas instancias estén operativas antes de alcanzar puntos críticos de carga

2. Migración a arquitectura basada en contenedores:

Evaluar el uso de servicios como Cloud Run o Kubernetes (GKE)

- Escalamiento horizontal más rápido (en cuestión de segundos)
- Mayor eficiencia en la gestión de recursos
- Simplificación del ciclo de vida de las aplicaciones
- Mejor mantenibilidad y despliegues continuos

La solución con contenedores representa un enfoque más moderno y elástico, particularmente adecuado para cargas de trabajo variables.

Diagrama UML

Este diagrama UML representa un modelo de clases simplificado que describe la relación entre las entidades Usuario, Chat y Documento que interactúan para formar una estructura de gestión de chats y archivos. La clase Usuario incluye atributos como un identificador único, nombre de usuario y contraseña, junto con métodos para validar y gestionar contraseñas, lo que refleja un enfoque en la seguridad y autenticación. Los usuarios tienen una relación uno a muchos con la clase Chat, indicando que un usuario puede crear múltiples chats, cada uno identificado por un ID único y vinculado al usuario correspondiente. A su vez, la clase Documento está asociada a los chats, almacenando información como el nombre del archivo, el ID del chat al que pertenece y una lista de fragmentos (chunking). En conjunto, el diagrama representa un modelo para un sistema de mensajería donde los usuarios interactúan mediante chats con capacidad de carga de documentos.

